Cahier des charges

CESIZen

Documentation Technique

Commanditaire	Ministère de la Santé et de la Prévention
Chef de Projet	Anaïs KAJJAJ
Lien vers le repository Git	https://github.com/Anaderis/cesizen-python
Formation	CESI - Bachelor Concepteur Développeur d'Applications
Date	01/04/2026

CESI
ÉCOLE D'INGÉNIEURS

MINISTÈRE
DE LA SANTÉ
ET DE LA PRÉVENTION
Liberté
Égalité
Fraternité

Sommaire

Sommaire **2**
Modèle de Données **3**
1.1 Les tables 3
1.2 Diagramme MCD 4
1.3 Diagramme MLD 4
Comparatif des solutions techniques **5**
2.1 Contexte et objectifs 5
2.2 Architectures étudiées 5
2.3 Critères d'évaluation 8
2.4 Tableau comparatif 9
Choix final de la solution **10**
3.1 Solution retenue 10
3.2 Justification du choix 10
3.3 Respect du pattern MVC 11
Guide d'installation de l'environnement de développement **12**
4.1 Prérequis 12
4.2 Récupérer le code source 13
4.3 Créer la base de données 13
4.4 Configurer les variables d'environnement 14
4.5 Installer et lancer le backend (FastAPI) 14
4.6 Installer et lancer le frontend (Vue.js) 15
4.7 Lancer les tests 15
4.8 Récapitulatif des URLs locales 16
Cahier de tests **16**
5.1 Environnement de test 16
5.2 Tests unitaires 17
5.3 Tests fonctionnels 18
5.4 Tests de non-régression 19
Procédure de validation et modèle de PV de recette **19**
6.1 Etapes de la procédure de validation 19
6.2 Modèle de Procès-Verbal de Recette 20
Annexes **21**
7.1 Diagramme MCD 21
7.2 Diagramme MLD 21
7.3 Stack technique du projet 22
7.4 Cahier de test 24
7.5 Procès-Verbal de Recette 41

PARTIE 1

Modèle de Données

La base de données de ce projet s'organise autour de 8 tables. MCD (Modèle conceptuel de données) représente les entités ainsi que leurs relations, tandis que le MLD représente l'utilisation concrète dans la base de données PostGreSQL avec les clés primaires, les clés étrangères et les tables de liaison.

1.1 Les tables

Le tableau ci-dessous présente les 6 tables principales et 2 tables de liaison :

Table	Clé primaire	Clés étrangères	Description
utilisateur	id	role_id → role	Toute personne ayant un compte sur l'application (utilisateur ou administrateur)
role	id		Définit le niveau d'accès d'un utilisateur (utilisateur standard ou administrateur)
article_sante	id	category_id	Articles informatifs sur la santé mentale, consultables par tous
category	id		Permet de classer les articles et les activités par thème
activity	id	category_id, format_id	Activités proposées aux utilisateurs pour gérer leur stress, peut être placé en favori si l'utilisateur est

Table	Clé primaire	Clés étrangères	Description
			connecté
format	id		Définit le type de support d'une activité (vidéo, PDF, lien externe, etc.)
favorites	table jointe : user_id + activity_id	user_id, activity_id	Table de liaison favoris
log	id	user_id → user	Historique des actions des administrateurs

Trois points sont également à prendre en compte :

- La table jointe “Favorites” fait le lien entre l’utilisateur et une activité qu’il a ajoutée en favori, elle a donc deux clés étrangères activity_id et user_id.
- Les tables “Category” et “Format” sont utilisées à la fois par les articles et les activités, afin de pouvoir les filtrer : par support (PDF, vidéo...) tandis que Category fait référence à un thème (Burn-out, dépression, méditation...).
- La table “log” permet d’effectuer le suivi des actions effectuées par un administrateur et de les télécharger, la table est donc rattachée au user_id.

1.2 Diagramme MCD

Le diagramme MCD est disponible en Annexe.

1.3 Diagramme MLD

Le diagramme MCD est disponible en Annexe.

PARTIE 2

Comparatif des solutions techniques

2.1 Contexte et objectifs

Avant de démarrer le développement de CESIZen, il a fallu choisir les technologies à utiliser. Ce choix est important car il est la base de l'architecture et de la manière dont l'application sera construite, testée et maintenue. Les principales contraintes étaient :

- L'application doit être accessible à tout le monde (visiteurs, utilisateurs, administrateurs)
- Les données des utilisateurs doivent être protégées par une connexion sécurisée
- Un espace d'administration doit permettre de gérer les contenus et les comptes
- L'application doit communiquer via une API entre le serveur et l'interface
- Des tests automatisés doivent garantir le bon fonctionnement à chaque modification

2.2 Architectures étudiées

Architecture A — Django (Python) + Django Templates

Django est un framework Python complet qui permet la gestion de la base de données, la connexion des utilisateurs, l' interface administrateur, ainsi que la génération des pages HTML côté serveur. Il n'est pas nécessaire d'ajouter de bibliothèques tierces pour démarrer un projet.

Fiche technique — Architecture A	
Backend	Django (Python)
Frontend	Django Templates
Base de données	PostgreSQL

Fiche technique — Architecture A	
Authentification	Sessions intégrées à Django

Points forts	Points faibles
- Très rapide à mettre en place grâce aux nombreux outils intégrés - Framework stable, avec une documentation très complète - Sécurité robuste par défaut	- Les pages sont entièrement générées par le serveur et rechargées à chaque changement à l'écran - Moins adapté si l'on souhaite proposer une application mobile

Architecture B — FastAPI (Python) + Vue.js 3

FastAPI est un framework Python conçu pour créer des APIs. Vue.js 3 permet de construire l'interface directement dans le navigateur, sans rechargement de page à chaque action effectuée.

Fiche technique — Architecture B	
Backend	FastAPI (Python) + SQLAlchemy
Frontend	Vue.js 3
Base de données	PostgreSQL
Authentification	JWT (JSON Web Tokens)

Points forts	Points faibles
- Très performant, parmi les plus rapides en Python - Une documentation de l'API est générée automatiquement - La navigation ne recharge pas la page entière	- Nécessite de gérer deux projets distincts (backend et frontend), - Certaines fonctionnalités avancées nécessitent plus de configuration manuelle

Architecture C — Node.js (Express) + React

Express.js est un framework pour Node.js permettant de créer des APIs en JavaScript. React est une bibliothèque Meta pour construire des interfaces web côté navigateur.

Fiche technique — Architecture C	
Backend	Node.js + Express.js
Frontend	React
Base de données	PostgreSQL
Authentification	JWT

Points forts	Points faibles
- Un seul langage est utilisé pour le front et le back-end - React est très utilisé dans l'industrie, de nombreuses ressources sont disponibles - Interface fluide et réactive, sans rechargement de page	- Express est minimaliste : il faut configurer manuellement beaucoup d'éléments (validation, sécurité, gestion des erreurs) - La sécurité par défaut est moins complète que celle de Django ou FastAPI - React a une courbe d'apprentissage plus difficile que Vue.js

2.3 Critères d'évaluation

Chaque architecture est notée de 1 à 5 pour chacun des critères ci-dessous. Un poids est attribué à chaque critère selon son importance dans le projet CESIZen.

#	Critère	Description	Poids
1	**Support et communauté**	Ancienneté du framework, communauté, qualité de la documentation	15 %
2	**Rapidité de développement**	Facilité de prise en main, outils intégrés	20 %
3	**Performance**	Vitesse de réponse, gestion des utilisateurs	15 %
4	**Sécurité**	Protections intégrées, gestion des rôles, chiffrement	15 %
5	**Séparation interface / serveur**	Clarté de la séparation, le back-end et le front-end peuvent évoluer indépendamment	10 %
6	**Testabilité**	Facilité à écrire et automatiser des tests, CI/CD	15 %
7	**Qualité de l'interface utilisateur**	Fluidité, réactivité, modernité de l'expérience	10 %

2.4 Tableau comparatif

Critère	Poids	A — Django	B — FastAPI	C — Express
Support et communauté	15 %	5	4	4
Rapidité de développement	20 %	5	4	3
Performance	15 %	3	**5**	4
Sécurité	15 %	5	4	3
Séparation interface / serveur	10 %	2	**5**	5
Testabilité	15 %	4	**5**	4
Qualité de l'interface	10 %	2	**5**	5
Score	**100 %**	**3,75 / 5**	**4,55 / 5**	**3,85 / 5**

Classement final

Position	Architecture	Score
1er	**FastAPI + Vue.js 3**	**4,55 / 5**
2ème	Express + React	3,85 / 5
3ème	Django + Templates	3,75 / 5

PARTIE 3

Choix final de la solution

3.1 Solution retenue

L'architecture retenue pour le projet CESIZen est l'**Architecture B** : FastAPI (Python) + Vue.js 3, avec PostgreSQL comme base de données et une authentification par JWT. Cette solution obtient le meilleur score (**4,55 / 5**) et répond à l'ensemble des contraintes identifiées dans le cahier des charges.

Le détail de la stack technique est disponible en Annexe.

3.2 Justification du choix

Côté serveur : Pourquoi FastAPI ?

Critère	Description
Rapidité	FastAPI peut traiter de nombreuses requêtes simultanément car il fonctionne en mode asynchrone.
Documentation intégrée	Une documentation interactive est automatiquement générée elle liste toutes les routes et permet de les tester rapidement
Gestion des droits d'accès	Il est possible de définir facilement quelles routes sont accessibles aux utilisateurs selon leurs rôles
Gestion de la base de données	SQLAlchemy permet de manipuler la base de données en Python, sans écrire de requêtes SQL à la main
Tests	Python offre des outils de test très complets (pytest, httpx) qui

Critère	Description
	s'intègrent bien avec FastAPI.

Côté front-end : Pourquoi Vue.js ?

Critère	Description
Prise en main	Vue.js est reconnu pour être plus simple à apprendre que React ou Angular, et reste est assez puissant pour ce projet.
Navigation fluide	Avec Vue Router, le passage d'une page à l'autre est instantané côté navigateur, sans rechargement complet.
Réactivité	Vue.js met automatiquement à jour l'interface dès qu'une donnée change (par exemple, l'affichage d'une liste de favoris après un clic).
Backend indépendant	Le frontend communique avec l'API FastAPI uniquement via des requêtes HTTP. Les deux parties peuvent évoluer indépendamment.

3.3 Respect du pattern MVC

L'architecture retenue respecte le Design Pattern **MVC (Modèle / Vue / Contrôleur)**, qui sépare les trois rôles de l'application :

Couche	Rôle	Implémentation dans le projet
Modèle	Représente les données et leur structure	Modèles SQLAlchemy (app/models/) et schémas Pydantic (app/schemas/)

Couche	Rôle	Implémentation dans le projet
Vue	Ce que voit et utilise l'utilisateur	Composants Vue.js (frontend/src/views/)
Contrôleur	Traite les demandes et fait le lien entre les données et l'interface	Routes FastAPI (app/controllers/) et services (la logique métier) (app/services/)

PARTIE 4

Guide d'installation de l'environnement de développement

4.1 Prérequis

Avant de commencer, vérifiez que les logiciels suivants sont installés sur votre machine :

Logiciel	Version minimale	Utilité
Python	3.10 ou supérieur	Faire tourner le serveur FastAPI
Node.js	20 ou supérieur	Faire tourner l'interface Vue.js
PostgreSQL	14 ou supérieur	Base de données de l'application
Git	Toute version récente	Récupérer le code source

Pour vérifier qu'un logiciel est bien installé :

```
python --version

node --version

psql --version

git --version
```

4.2 Récupérer le code source

Action	Commande
Cloner le dépôt	git clone https://github.com/Anaderis/cesizen-python.git
Se placer dans le projet	cd cesizen-python

4.3 Créer la base de données

Ouvrez PostgreSQL (via pgAdmin ou en ligne de commande) et créez une base de données nommée cesizen :

Action	**Commande**
Créer la base	CREATE DATABASE cesizen;
Importer les données initiales	psql -U postgres -d cesizen -f app/static/sql/cesizen-0104.sql

4.4 Configurer les variables d'environnement

Les informations sensibles sont stockées dans un fichier .env qui n'est pas sur git par sécurité. Un fichier modèle .env.example est fourni. Copiez-le et renommez-le .env, puis renseignez vos propres valeurs PostGreSQL:

Variable	Valeur exemple	Description
SECRET_KEY	*clé secrète générée*	Clé secrète JWT
DATABASE_URL	postgresql://user:password@localhost:5432/cesizen	URL de connexion PostgreSQL

> Générer une clé secrète sécurisée via un terminal python (commande : python) :
>
> import secrets ; print(secrets.token_hex(32))

4.5 Installer et lancer le backend (FastAPI)

Action	**Commande**
Créer l'environnement virtuel	python -m venv pythonCesizen
Activer (Windows)	pythonCesizen\Scripts\activate
Activer (Mac / Linux)	source pythonCesizen/bin/activate
Installer les dépendances	pip install -r requirements.txt

Action	Commande
Lancer le serveur	uvicorn app.main:app --reload
Lancer les tests	pytest

4.6 Installer et lancer le frontend (Vue.js)

Action	Commande
Aller dans le dossier	cd frontend
Installer les dépendances	npm install
Lancer en développement	npm run dev
Build de production	npm run build

4.7 Lancer les tests

Pour vérifier que tout fonctionne correctement, revenez dans le dossier racine et exécutez :

Action	Commande
Lancer tous les tests	pytest
Lancer avec détails	pytest tests/ -v

Tous les tests doivent passer (affichage en vert). Si un test échoue, vérifiez que la base de données est bien configurée et que le serveur backend tourne correctement.

4.8 Récapitulatif des URLs locales

Ordre de démarrage

Pour utiliser CESIZen au quotidien, voici l'ordre à suivre à chaque démarrage :

- Activer l'environnement virtuel Python (pythonCesizen\Scripts\activate)
- Lancer le backend dans un terminal : uvicorn app.main:app --reload
- Lancer le frontend dans un autre terminal : cd frontend && npm run dev
- Ouvrir le navigateur sur **http://localhost:5173**

Service	URL	Description
Frontend Vue.js	http://localhost:5173	Interface utilisateur
API FastAPI	http://localhost:8000	API REST
Documentation Swagger	http://localhost:8000/docs	Interface interactive de l'API
PostgreSQL	localhost:5432	Base de données

PARTIE 5

Cahier de tests

5.1 Environnement de test

Les tests sont exécutés avec la commande **pytest** depuis le dossier racine du projet. Ils utilisent une base de données temporaire, distincte de la base de production, qui est créée au lancement des tests et supprimée à la fin.

Trois types de profils sont simulés pendant les tests :

Profil	Droits
Administrateur	Accès complet à toutes les fonctionnalités
Utilisateur connecté	Accès à son propre compte et à ses favoris
Visiteur (non connecté)	Accès aux contenus publics uniquement

5.2 Tests unitaires

L'objectif du test unitaire est de vérifier dans un environnement isolé (pas de serveur ou de base de données en cours d'exécution) qu'une partie du code spécifique fonctionne correctement, et à l'inverse, le test nous permettra d'isoler et d'identifier rapidement d'où provient l'erreur. On va ainsi simuler une "fausse" base en mémoire que l'on appelle un "mock".

Voici ci-dessous la liste des tests unitaires effectués :

ID	Scénario
TU-01	Validation du formulaire d'inscription
TU-02	Validation de l'URL d'une activité
TU-03	Authentification utilisateur
TU-04	Hachage des mots de passe

Scénarii de tests :
Le détail de chacun des scénarios de tests unitaires est disponible en Annexe.

5.3 Tests fonctionnels

Les tests fonctionnels valident le comportement de chaque route de l'API REST via pytest et httpx. Le backend doit être en cours d'exécution et la base de données initialisée avec les données de test.

ID	Scénario
TF-01	Connexion via l'API
TF-02	Création d'un compte utilisateur
TF-03	Gestion des utilisateurs (accès administrateur)
TF-04	Modification de profil
TF-05	Désactivation de compte
TF-06	Gestion des favoris
TF-07	Consultation et gestion des articles de santé
TF-08	Gestion des activités bien-être

Scénarii de tests :
Le détail de chacun des scénarios de tests fonctionnels est disponible en Annexe.

5.4 Tests de non-régression

Les tests de non-régression vérifient que les règles métier du cahier des charges sont bien respectées avant la mise en production. Ils doivent être rejoués intégralement à chaque livraison.

ID	Scénario
TNR-01	Actions non autorisées sur les contenus
TNR-02	Contrôle d'accès aux pages d'administration
TNR-03	Protection des comptes utilisateurs

> **Scénarii de tests** :
> Le détail de chacun des scénarios de tests de non-régression est disponible en Annexe.

PARTIE 6

Procédure de validation et modèle de PV de recette

6.1 Etapes de la procédure de validation

Etape 1 : Préparation

Avant de commencer les tests, s'assurer que :

- L'application est installée et fonctionnelle sur l'environnement de recette (voir section 5)
- La base de données est alimentée avec les données de test

- Les trois profils utilisateurs sont disponibles : administrateur, utilisateur connecté, visiteur

Etape 2 : Exécution des tests automatisés

Lancer la suite de tests automatisés depuis le dossier racine du projet :

```
pytest tests/ -v
```

- Tous les tests doivent passer (affichage en vert)
- En cas d'échec, noter le ou les tests concernés et corriger avant de poursuivre.

Etape 3 : Rédaction du PV de recette

À l'issue des tests, compléter le modèle de PV de recette (section 7.3) avec les résultats obtenus, puis le faire signer par le responsable du projet. Selon les résultats du PV de recette, une décision sera apportée :

- Tous les tests sont conformes : Validation, mise en production autorisée
- Anomalies mineures : Validation sous condition, des corrections sont à apporter avant livraison
- Anomalies bloquantes : Validation refusée, retour en développement.

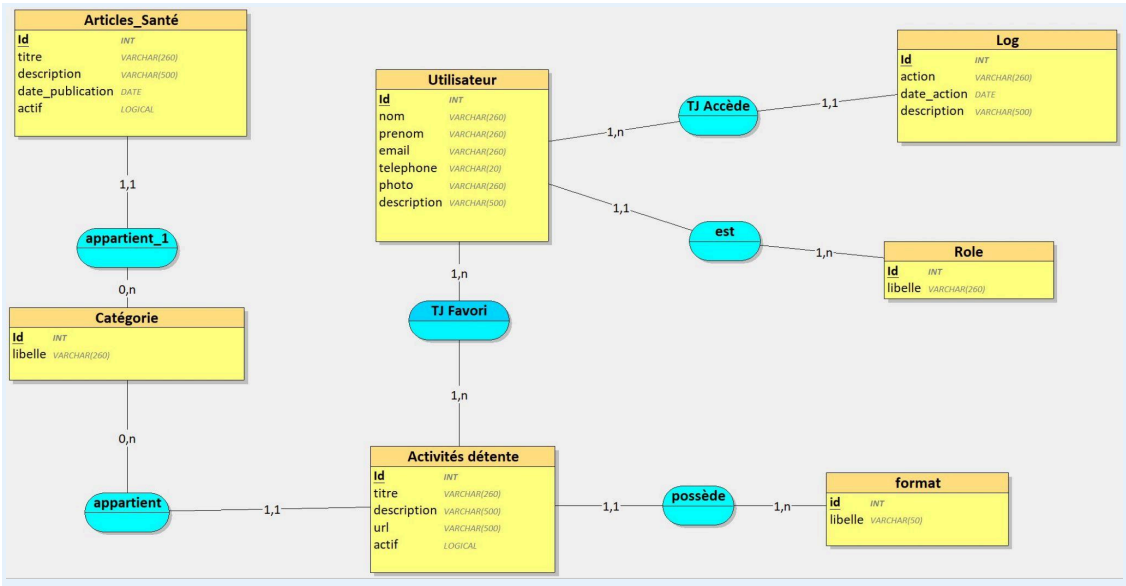
6.2 Modèle de Procès-Verbal de Recette

Le modèle de Procès-Verbal de Recette est disponible en Annexe.

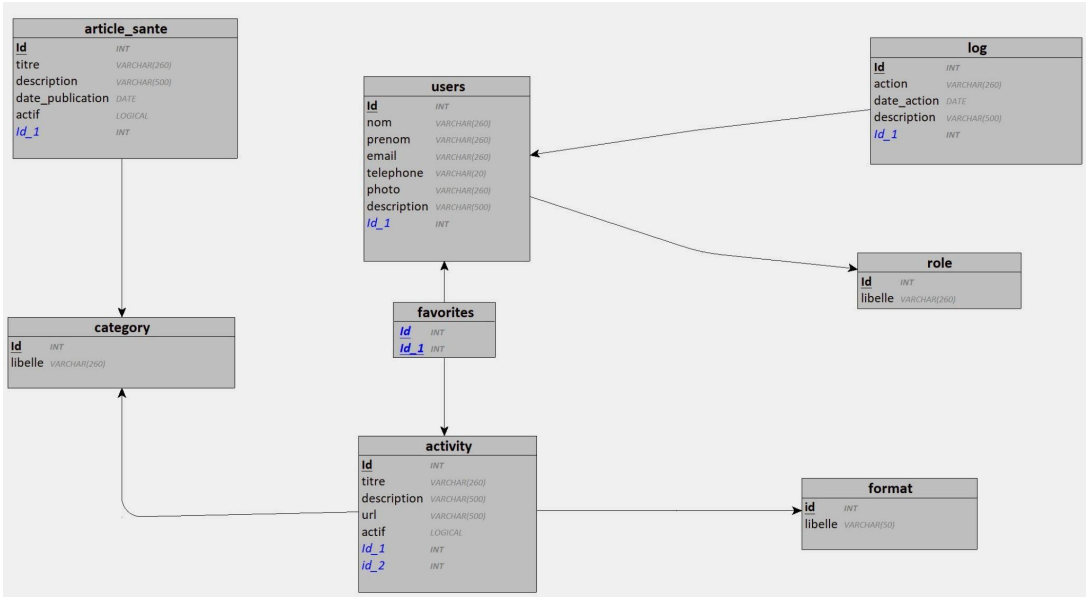
PARTIE 7

Annexes

7.1 Diagramme MCD



7.2 Diagramme MLD



7.3 Stack technique du projet

Stack technique du Back-End

Outil	Version	Description
FastAPI	0.115+	Framework Python asynchrone pour API REST
SQLAlchemy	2.0+	Interaction avec la base de données
Pydantic	2.0+	Validation des données
passlib (bcrypt)	1.7+	Hachage des mots de passe
uvicorn		Serveur pour lancer FastAPI
pytest		Tests automatisés (unitaires et fonctionnels)

Stack technique du Front-End

Technologie	Version	Description
Vue 3	3.5	Framework JavaScript progressif
Vite	5	Outil de build rapide
Vue Router	4.5	Gestion des routes
Pinia	3.0	Gestion d'état global

Stack technique de la Base de Données

Technologie	Version	Description
PostgreSQL	14+	Base de données relationnelle principale
PGAdmin		Interface graphique d'administration

7.4 Cahier de test

Configuration commune

Back-end	http://localhost:8000
Front-end	http://localhost:5173
Compte de test	lr@cesizen.fr / Admin123!

Tests unitaires

TU-01 — Validation du formulaire d'inscription

Nom du test	Validation du formulaire d'inscription
ID du test	TU-01
Objectif	Vérifier que le schéma Pydantic UserCreate accepte uniquement les données valides et rejette les autres données (email, mot de passe, nom).
Conditions préalables	Environnement virtuel Python activé. Dépendances installées (pip install -r requirements.txt).
Configurations	Python 3.10+ / Pydantic 2.0+ / pytest. Pas de connexion à la base de données requise.

ID du test	Etapes de test	Résultats Attendus
TU-01-01	1. Lancer pytest tests/unit/ 2. Tester UserCreate avec des	Données valides : objet créé sans erreur.

	données valides	
TU-01-02	1. Lancer pytest tests/unit/ 2. Tester avec un email en majuscules	Email en majuscules : converti automatiquement en minuscules.
TU-01-03	1. Lancer pytest tests/unit/ 2. Test avec un email invalide	Email invalide : une ValidationError est levée
TU-01-04	1. Lancer pytest tests/unit/ 2. Tester avec un mot de passe inférieur à 8 caractères	Mot de passe trop court / sans majuscule / sans chiffre / sans caractère spécial : une ValidationError est levée
TU-01-05	1. Lancer pytest tests/unit/ 2. Tester avec un mot de passe sans majuscule, sans chiffre, sans caractère spécial	Mot de passe trop court / sans majuscule / sans chiffre / sans caractère spécial : ValidationError est levée
TU-01-06	1. Lancer pytest tests/unit/ 2.Tester avec un nom vide ou composé d'espaces	Nom vide : ValidationError levée.
TU-01-07	1. Lancer pytest tests/unit/ 2. Instancier avec un nom entouré d'espaces	Nom avec espaces : espaces supprimés automatiquement.

TU-02 — Validation de l'URL d'une activité

Nom du test	Validation de l'URL d'une activité
ID du test	TU-02
Objectif	Vérifier que le schéma ActivityCreate accepte les formats

	autorisés (YouTube, fichiers internes) et rejette les valeurs non conformes.
Conditions préalables	Environnement virtuel Python activé. Dépendances installées.
Configurations	Python 3.10+ / Pydantic 2.0+ / pytest. Pas de connexion à la base de données requise.

ID du test	Etapes de test	Résultats Attendus
TU-02-01	1. Lancer pytest tests/unit/ 2. Soumettre une URL YouTube complète	URLs YouTube et chemins internes (PDF, audio) : acceptés sans erreur.
TU-02-02	1. Lancer pytest tests/unit/ 2. Soumettre un lien court YouTube	Accepté
TU-02-03	1. Lancer pytest tests/unit/ 2. Soumettre un chemin vers un fichier PDF interne	Accepté
TU-02-04	1. Lancer pytest tests/unit/ 2. Soumettre un texte quelconque sans format valide	Texte sans format valide : ValidationError levée.

TU-03 — Authentification utilisateur

Nom du test	Authentification utilisateur
ID du test	TU-03
Objectif	Vérifier que la fonction d'authentification retourne un token valide pour des identifiants corrects et lève une erreur pour des identifiants incorrects.
Conditions préalables	Environnement virtuel activé. Mock de la base de données configuré dans le fichier de test.
Configurations	Python 3.10+ / pytest / passlib (bcrypt) / python-jose. Base simulée via fixture pytest.

ID du test	Etapes de test	Résultats Attendus
TU-03-01	1. Lancer pytest tests/unit/ 2. Appeler la fonction d'authentification avec un email inconnu	Email inconnu : connexion refusée, exception levée.
TU-03-02	1. Lancer pytest tests/unit/ 2.Appeler avec un email connu mais un mauvais mot de passe	Mauvais mot de passe : connexion refusée, exception levée.
TU-03-03	1. Lancer pytest tests/unit/ 2. Appeler avec des identifiants corrects	Identifiants corrects : token JWT retourné

TU-04 — Hachage des mots de passe

Nom du test	Hachage des mots de passe
ID du test	TU-04
Objectif	Vérifier que le mot de passe est stocké sous forme hachée (bcrypt) et non en clair en base de données.
Conditions préalables	Environnement virtuel activé. Fixture de base de données de test disponible.
Configurations	Python 3.10+ / passlib (bcrypt) / pytest.

ID du test	Etapes de test	Résultats Attendus
TU-04-01	1. Créer un utilisateur avec le mot de passe Test1234!	Le champ hashed_password ne contient pas la valeur Test1234!.
TU-04-02	2. Récupérer l'enregistrement correspondant en base	La valeur commence par $2b$ (signature bcrypt).
TU-04-03	3. Comparer la valeur stockée dans le champ hashed_password avec le mot de passe en clair	La fonction verify_password retourne True pour le mot de passe correct.

Tests fonctionnels

TF-01 — Connexion via l'API

Nom du test	Connexion via l'API
ID du test	TF-01
Objectif	Vérifier que la route POST /auth/login retourne un token JWT pour des identifiants valides et renvoie les bons codes d'erreur sinon.
Conditions préalables	Backend lancé. Comptes lk@cesizen.fr et anais@mail.com présents en base.
Configurations	FastAPI 0.115+ / pytest / httpx. Base PostgreSQL de test initialisée.

ID du test	Etapes de test	Résultats Attendus
TF-01-01	1. Envoyer POST /auth/login avec lk@cesizen.fr / Admin123!	Identifiants valides : 200 : token JWT retourné.
TF-01-02	2. Envoyer POST /auth/login avec lk@cesizen.fr / mauvais_mdp	Mauvais mot de passe : 401 : accès refusé.
TF-01-03	3. Envoyer POST /auth/login avec un corps incomplet (email manquant)	Corps incomplet : 422 : données manquantes.

TF-02 — Création d'un compte utilisateur

Nom du test	Création d'un compte utilisateur
ID du test	TF-02
Objectif	Vérifier que la route publique POST /users/ crée un compte avec des données valides et rejette les données invalides.
Conditions préalables	Backend lancé. Aucune authentification requise pour cet endpoint.
Configurations	FastAPI 0.115+ / pytest / httpx. Base PostgreSQL de test.

ID du test	Etapes de test	Résultats Attendus
TF-02-01	1. Envoyer POST /users/ avec des données valides (email, nom, mot de passe conforme)	Données valides : 200 : compte créé, objet utilisateur retourné.
TF-02-02	2. Envoyer avec un email invalide (ex : pasunemail)	Email invalide : 422, erreur de validation.
TF-02-03	3. Envoyer avec un mot de passe ne respectant pas les règles de sécurité (ex : 1234)	Mot de passe faible : 422, erreur de validation.

TF-03 — Gestion des utilisateurs (accès admin)

Nom du test	Gestion des utilisateurs — accès admin
ID du test	TF-03
Objectif	Vérifier que la liste des utilisateurs est accessible uniquement par un administrateur, et que la suppression d'un compte fonctionne correctement selon le rôle.
Conditions préalables	Backend lancé. Comptes admin et utilisateur présents en base. Tokens JWT des deux rôles disponibles.
Configurations	FastAPI 0.115+ / pytest / httpx. Base PostgreSQL de test.

ID du test	Etapes de test	Résultats Attendus
TF-03-01	1. Envoyer GET /users/ avec le token admin	Admin : 200 , liste retournée.
TF-03-02	2. Envoyer GET /users/ avec le token utilisateur → vérifier le refus	Utilisateur : 403 , accès interdit.
TF-03-03	3. Envoyer GET /users/ sans token	Visiteur : 401 , non authentifié.
TF-03-04	4. Envoyer DELETE /users/{id} avec le token admin	Admin - suppression : 200, compte supprimé.
TF-03-05	5. Envoyer DELETE /users/{id} sans token	Visiteur - suppression: 401 , non authentifié.

TF-04 — Modification de profil

Nom du test	Modification de profil
ID du test	TF-04
Objectif	Vérifier qu'un utilisateur peut modifier uniquement son propre compte, qu'un administrateur peut modifier n'importe quel compte, et qu'un utilisateur ne peut pas modifier le compte d'autrui.
Conditions préalables	Backend lancé. Deux comptes utilisateurs distincts et un compte admin en base. Tokens JWT disponibles.
Configurations	FastAPI 0.115+ / pytest / httpx. Base PostgreSQL de test.

ID du test	Etapes de test	Résultats Attendus
TF-04-01	1. Envoyer PUT /users/{id} avec le token du propriétaire du compte	Propriétaire : 200, compte mis à jour.
TF-04-02	2. Envoyer PUT /users/{id_autre} avec le token d'un autre utilisateur	Autre utilisateur : 403, accès interdit.
TF-04-03	3. Envoyer PUT /users/{id} avec le token admin	Admin : 200, compte mis à jour.
TF-04-04	4. Envoyer PUT /users/{id} sans token	Visiteur : 401, non authentifié.

TF-05 — Désactivation de compte

Nom du test	Désactivation de compte
ID du test	TF-05
Objectif	Vérifier que la désactivation d'un compte est soumise aux mêmes règles d'accès que la modification : autorisée pour le propriétaire et l'admin, interdite pour les autres.
Conditions préalables	Backend lancé. Comptes utilisateurs et admin en base. Tokens JWT disponibles.
Configurations	FastAPI 0.115+ / pytest / httpx. Base PostgreSQL de test.

ID du test	Etapes de test	Résultats Attendus
TF-05-01	1. Envoyer PATCH /users/{id}/deactivate avec le token du propriétaire	Propriétaire : 200, compte désactivé.
TF-05-02	2. Envoyer avec le token d'un autre utilisateur	Autre utilisateur : 403, accès interdit.
TF-05-03	3. Envoyer avec le token admin	Admin : 200, compte désactivé.
TF-05-04	4. Envoyer sans token	Visiteur : 401, non authentifié.

TF-06 — Gestion des favoris

Nom du test	Gestion des favoris
ID du test	TF-06
Objectif	Vérifier qu'un utilisateur connecté peut consulter ses favoris et qu'un visiteur non authentifié n'y a pas accès.
Conditions préalables	Backend lancé. Compte utilisateur avec au moins un favori en base. Token JWT disponible.
Configurations	FastAPI 0.115+ / pytest / httpx. Base PostgreSQL de test.

ID du test	Etapes de test	Résultats Attendus
TF-06-01	1. Envoyer GET /users/me/favorites avec un token utilisateur valide	Connecté : 200, liste des favoris retournée.
TF-06-02	Envoyer GET /users/me/favorites sans token	Visiteur : 401, non authentifié.

TF-07 — Consultation et gestion des articles de santé

Nom du test	Consultation et gestion des articles de santé
ID du test	TF-07
Objectif	Vérifier que les articles sont consultables publiquement, qu'un article inexistant retourne une 404, et que la modification est réservée à l'administrateur.
Conditions préalables	Backend lancé. Au moins un article en base. Tokens admin et utilisateur disponibles.
Configurations	FastAPI 0.115+ / pytest / httpx. Base PostgreSQL de test.

ID du test	Etapes de test	Résultats Attendus
TF-07-01	1. Envoyer GET /articles/ sans token	Liste publique : 200, liste retournée.
TF-07-02	2. Envoyer GET /articles/{id} avec un id valide	ID valide : 200, article retourné.
TF-07-03	3. Envoyer GET /articles/99999 (inexistant)	Article inexistant : 404, non trouvé.
TF-07-04	4. Envoyer PUT /articles/{id} avec le token admin	Modification admin : 200, article mis à jour.
TF-07-05	5. Envoyer PUT /articles/{id} avec le token utilisateur	Modification utilisateur : 403 accès interdit.

TF-08 — Gestion des activités bien-être

Nom du test	Gestion des activités bien-être
ID du test	TF-08
Objectif	Vérifier les droits d'accès sur les routes activités : consultation publique des activités actives, gestion complète réservée à l'administrateur, rejet des données invalides.
Conditions préalables	Backend lancé. Activités actives et inactives en base. Tokens admin, utilisateur disponibles.
Configurations	FastAPI 0.115+ / pytest / httpx. Base PostgreSQL de test.

ID du test	Etapes de test	Résultats Attendus
TF-08-01	1. Envoyer GET /activities/ sans token	Liste publique : 200, activités actives uniquement.
TF-08-02	2. Envoyer GET /activities/all avec token admin	Liste complète admin : 200, toutes les activités.
TF-08-03	3. Envoyer GET /activities/all avec token utilisateu	Liste complète utilisateur : 403, accès interdit.
TF-08-04	4. Envoyer POST /activities/ avec token admin et données valides	Création admin : 200, activité créée.
TF-08-05	5. Envoyer POST /users/ avec token utilisateur	Création utilisateur : 403, accès interdit.
TF-08-06	6. Envoyer POST /activities/ sans	Création visiteur : 401, non

	token	authentifié.
TF-08-07	7. Envoyer PATCH /activities/{id}/deactivate avec token admin	Désactivation admin : 200, activité désactivée.
TF-08-08	8. Envoyer PUT /activities/{id} avec token admin	Modification admin : 200, activité mise à jour.
TF-08-09	9. Envoyer PUT /activities/{id} avec token utilisateur	Modification utilisateur : 403, accès interdit.
TF-08-10	10. Envoyer PUT /activities/{id} avec une URL invalide	URL invalide : 422, données invalides.

Tests de non-régression

TNR-01 — Actions non autorisées sur les contenus

Nom du test	Actions non autorisées sur les contenus
ID du test	TNR-01
Objectif	Vérifier que la création et la suppression d'articles, ainsi que la suppression d'activités, ne sont pas exposées via l'API, conformément aux spécifications du projet.
Conditions préalables	Backend lancé. Token admin disponible.
Configurations	FastAPI 0.115+ / pytest / httpx. À rejouer après chaque modification du code des routes.

ID du test	Etapes de test	Résultats Attendus
TNR-01-01	1. Envoyer POST /articles/ avec token admin	Création d'article : 405, méthode non autorisée.
TNR-01-02	Envoyer DELETE /articles/{id} avec token admin	Suppression d'article : 405, méthode non autorisée.
TNR-01-03	3. Envoyer DELETE /activities/{id} avec token admin	Suppression d'activité : 405, méthode non autorisée.

TNR-02 — Contrôle d'accès aux pages d'administration

Nom du test	Contrôle d'accès aux pages d'administration
ID du test	TNR-02
Objectif	Vérifier qu'aucune route d'administration n'est accessible par un utilisateur standard ou un visiteur non authentifié, quelle que soit la modification apportée au code.
Conditions préalables	Backend lancé. Token utilisateur disponible. Aucun token pour le cas visiteur.
Configurations	FastAPI 0.115+ / pytest / httpx. À rejouer après chaque modification des dépendances d'authentification.

ID du test	Etapes de test	Résultats Attendus
TNR-02-01	1. Envoyer GET /users/ avec token utilisateur	Token utilisateur sur toute route admin : 403, accès interdit.
TNR-02-02	2. Envoyer GET /users/ sans token	Aucun token sur toute route protégée : 401, non authentifié.
TNR-02-03	3. Envoyer GET /activities/all avec token utilisateur	Token utilisateur sur toute route admin : 403, accès interdit.
TNR-02-04	4. Envoyer GET /logs/ avec token utilisateur	Token utilisateur sur toute route admin : 403, accès interdit.
TNR-02-05	5. Envoyer GET /logs/ sans token	Aucun token sur toute route protégée : 401, non authentifié.

TNR-03 — Protection des comptes utilisateurs

Nom du test	Protection des comptes utilisateurs
ID du test	TNR-03
Objectif	Vérifier qu'un utilisateur ne peut en aucun cas modifier ou désactiver le compte d'une autre personne, même en forçant l'identifiant dans l'URL.
Conditions préalables	Backend lancé. Deux comptes utilisateurs distincts en base. Tokens JWT des deux utilisateurs disponibles.
Configurations	FastAPI 0.115+ / pytest / httpx. À rejouer après chaque modification des services utilisateurs.

ID du test	Etapes de test	Résultats Attendus
TNR-03-01	1. Récupérer l'ID de l'utilisateur B 2. Envoyer PUT /users/{id_B} avec le token de l'utilisateur A	Modification du compte d'autrui : 403 , accès interdit.
TNR-03-02	1. Récupérer l'ID de l'utilisateur B 2. Envoyer PATCH /users/{id_B}/deactivate avec le token de l'utilisateur A	Désactivation du compte d'autrui : 403 , accès interdit.

7.5 Procès-Verbal de Recette

CESIZen - v1.0 (Prototype)

1. Identification

Projet	CESIZen — Plateforme de bien-être mental
Auteur	Anaïs Kajjaj
Version testée	1.0 (prototype)
Environnement	Local (Python + PostgreSQL)
Date de recette	
Testeur(s)	

2. Périmètre

Ce PV atteste que les fonctionnalités livrées dans le cadre du prototype v1.0 ont été vérifiées et sont conformes aux spécifications fonctionnelles.

Comptes de test utilisés :

Rôle	Email	Mot de passe
Administrateur	lk@cesizen.fr	Admin123!
Utilisateur connecté	anais@gmail.com	Anais123!
Visiteur (non authentifié)		

Services testés :

Application	URL locale
Frontend Vue.js	http://localhost:5173
API REST (Swagger)	http://localhost:8000/docs

3. Résumé des résultats

Type de tests	Nombre de groupes	Groupes réussis	Groupes échoués	Observations
Tests unitaires	4 (TU-01 à TU-04)			
Tests fonctionnels	8 (TF-01 à TF-08)			
Tests de non-régression	3 (TNR-01 à TNR-03)			
Total	**12**			

4. Résultats détaillés

Tests unitaires

Exécutés via pytest. La base de données est simulée (mock). Les tests vérifient les schémas Pydantic de façon isolée.

ID	Nom du test	Description	Statut	Observations
TU-01	Validation du formulaire d'inscription	Vérifier que le schéma Pydantic UserCreate accepte uniquement les données valides et rejette les autres données (email, mot de passe, nom). (7 sous-tests).		
TU-02	Validation de l'URL d'une activité	Vérifier que le schéma ActivityCreate accepte les formats autorisés (YouTube, fichiers internes) et rejette les valeurs non conformes. (4 sous-tests).		
TU-03	Connexion utilisateur	Vérifier que la fonction d'authentification retourne un token valide pour des identifiants corrects et lève une erreur pour des identifiants incorrects. (3 sous-tests).		
TU-04	Sécurité des mots de passe	Vérifie que le mot de passe est haché et illisible en base de données après création du compte (3		

ID	Nom du test	Description	Statut	Observations
		sous-tests).		

Tests fonctionnels

Exécutés via pytest et httpx. Chaque test envoie une requête HTTP réelle à l'API FastAPI et vérifie le code de retour et les données.

ID	Nom du test	Description	Statut	Observations
TF-01	Authentification	Vérifier que la route POST /auth/login retourne un token JWT pour des identifiants valides et renvoie les bons codes d'erreur sinon. (3 sous-tests).		
TF-02	Création d'un utilisateur	Vérifier que la route publique POST /users/ crée un compte avec des données valides et rejette les données invalides. (3 sous-tests).		
TF-03	Gestion des utilisateurs — accès admin	Vérifier que la liste des utilisateurs est accessible uniquement par un administrateur, et que la suppression d'un compte fonctionne correctement selon le rôle.(5 sous-tests).		
TF-04	Modification de profil	Vérifier qu'un utilisateur peut modifier uniquement son propre compte, qu'un administrateur peut modifier n'importe quel compte, et qu'un utilisateur ne peut pas modifier le compte d'autrui. (4 sous-tests).		

ID	Nom du test	Description	Statut	Observations
TF-05	Désactivation de compte	Vérifier que la désactivation d'un compte est soumise aux mêmes règles d'accès que la modification : autorisée pour le propriétaire et l'admin, interdite pour les autres. (4 sous-tests).		
TF-06	Gestion des favoris	Vérifier qu'un utilisateur connecté peut consulter ses favoris et qu'un visiteur non authentifié n'y a pas accès. (2 sous-tests).		
TF-07	Consultation et gestion des articles de santé	Vérifier que les articles sont consultables publiquement, qu'un article inexistant retourne une 404, et que la modification est réservée à l'administrateur. (5 sous tests)		
TF-08	Gestion des activités bien-être	Vérifier les droits d'accès sur les routes activités : consultation publique des activités actives, gestion complète réservée à l'administrateur, rejet des données invalides.		

Tests de non-régression

Vérifient que les règles métier fondamentales (contrôle des accès, actions interdites) sont maintenues après chaque évolution du code.

ID	Nom du test	Description	Statut	Observations
TNR-01	Actions non autorisées sur les contenus	Vérifier que la création et la suppression d'articles, ainsi que la suppression d'activités, ne sont pas exposées via l'API, conformément aux spécifications du projet. (3 sous-tests).		
TNR-02	Contrôle d'accès aux pages d'administration	Vérifier qu'aucune route d'administration n'est accessible par un utilisateur standard ou un visiteur non authentifié, quelle que soit la modification apportée au code. (5 sous-tests).		
TNR-03	Protection des comptes utilisateurs	Vérifier qu'un utilisateur ne peut en aucun cas modifier ou désactiver le compte d'une autre personne, même en forçant l'identifiant dans l'URL.		

5. Anomalies relevées

ID anomalie	Description	Module concerné	Priorité	Statut

Priorités : Bloquante / Majeure / Mineure

6. Conclusion

Les fonctionnalités testées dans le cadre de cette recette sont conformes aux spécifications du prototype v1.0 de l'application CESIZen.

Décision :

☐ Validé sans réserve ☐ Validé avec réserves ☐ Non validé

Réserves éventuelles :

Signature du testeur	Validation du responsable
Nom :	Nom :
Date :	Date :
Signature :	Signature :